EVENT MANAGEMENT SYSTEM

Project Documentation & Progress Report

Sprint 1 Complete | Sprint 2 In Progress | 8 of 14 Days Done

Student Name	Abdul Arfath
Registration No	P05MG24S038006
Department	PG Department of Computer Science
Institution	Mahatma Gandhi Memorial College, Udupi – 576102
Internship Organisation	Global Zenith Tech, Mandavi Prince Palace, Udupi 576102
Project Title	Event Management System (EMS)
Academic Year	2025–2026
Document Date	March 2026
Development Status	Sprint 1 Complete | Sprint 2 (50% done)
GitHub Branch	develop (protected main branch)

1. Project Overview

1.1 Problem Statement

Managing B2B events today requires maintaining a patchwork of disconnected tools — separate platforms for delegate registration, sponsor communication, onsite check-in, session scheduling, and virtual hosting. Because these systems operate in silos, event data remains fragmented with no consolidated view. Organizers fall back on spreadsheets for check-ins, chase sponsor deliverables manually, and constantly shuttle information between platforms.

Participants face equal friction, needing to navigate several platforms simply to register, view an agenda, or join a session. When events go live, the lack of centralised real-time visibility leaves organizers unable to access immediate attendance figures or gauge engagement.

1.2 Solution

The EMS is a unified, role-based web platform built to address these gaps. It consolidates the complete B2B event lifecycle — from pre-event planning through onsite execution to post-event analysis — into a single locally deployable application.

1.3 Objectives

- Provide organizers a single platform to create, configure, publish, and manage B2B events of any size and format
- Enable seamless delegate registration with QR-code-based ticketing and PayPal-integrated payment processing
- Deliver a real-time organizer dashboard with live visibility into registrations, revenue, and check-ins
- Support private local-server deployment via Docker for full organisational data sovereignty
- Automate post-event workflows including attendance certificates, feedback surveys, and sponsor outcome reports

1.4 Scope

The platform serves four user roles within a single unified environment:

Role	Who they are	What they can do
Organizer	Event planners who create and manage events	Create events, configure tickets, view registrations, manage agenda, check in attendees, view reports
Attendee	End-users who register and attend events	Discover events, register, pay, access agenda, view tickets with QR codes
Admin	Platform-level superuser	Manage users, oversee system settings, handle support
Public / Guest	Unauthenticated visitors	Browse published events, view event details

2. Technology Stack

Category	Technology	Role in the project
Backend	Python 3.11 + Flask	Core application logic, routing, request handling, blueprint architecture
Database	Firebase Firestore	NoSQL document database — stores users, events, venues, registrations, tickets
Authentication	Firebase Auth	Email/password authentication, user identity, UID-based access control
File Storage	Firebase Storage	QR code PNGs, event cover images, attendance certificates (Sprint 2+)
Frontend	Bootstrap 5 + Jinja2	Responsive HTML templates, component styling, mobile support
Icons	Bootstrap Icons	UI icons throughout — calendar, QR scanner, ticket, building, etc.
QR Codes	qrcode[pil] library	Generates HMAC-signed QR code PNG images for attendee tickets
Payments	PayPal Checkout SDK	Processes ticket payments, handles webhooks, IPN confirmation (Sprint 2)
Email	SendGrid	Sends confirmation emails with QR code images attached (Sprint 2)
Security	Flask-WTF + HMAC	CSRF protection on all forms; HMAC signature verification on QR codes
Deployment	Docker + Docker Compose	Containerised deployment — consistent across dev and production environments
Version Control	Git + GitHub	Source control, branch strategy (main/develop), release tagging

3. System Architecture

3.1 Application Structure

The application follows the Flask Blueprint pattern — each functional area (auth, events, venues, tickets, check-in, public pages) is a separate blueprint module. This keeps code organised and allows independent development of each feature area.

File / Folder	Purpose
app.py	Entry point — initialises Flask, registers all blueprints, starts the server
app/firebase_config.py	Firebase initialisation module — connects to Firestore, Auth, Storage
app/decorators.py	Route guards — login_required and role_required decorators
app/routes/auth.py	Authentication routes — sign up, login, logout, forgot password
app/routes/public.py	Public pages — event listing and event detail pages
app/routes/events.py	Organizer event management — create, edit, publish, cancel events
app/routes/venues.py	Organizer venue management — add, edit, delete venues
app/routes/tickets.py	Ticket type management — add, edit, delete, toggle ticket types per event
app/routes/checkin.py	Check-in system — QR scanner API, manual check-in, check-in log
app/routes/organizer.py	Organizer dashboard — aggregated metrics across all events
app/utils/validators.py	Reusable form validators for venues, events, and ticket types
app/utils/qr_utils.py	QR code generation and HMAC payload verification utilities
templates/base.html	Master layout — navbar (role-aware), flash messages, footer
templates/public/	Event listing and event detail pages (public-facing)
templates/auth/	Login, signup, forgot password pages
templates/organizer/	Dashboard, event form, venue form, ticket form, check-in scanner
static/css/ems.css	Custom styles — blue & white corporate theme on Bootstrap 5
secrets/firebase-key.json	Firebase service account key (Docker volume, never in Git)
Dockerfile	Container build recipe — Python 3.11 slim, pip installs, EXPOSE 5000
docker-compose.yml	Service definition — port mapping, env file, secrets volume mount
.env	Environment variables — Firebase, PayPal, SendGrid, QR secret (never in Git)

3.2 Request Flow

Every HTTP request follows this path through the application:

- Browser sends request → Docker routes to Flask on port 5000
- Flask matches the URL to a blueprint route function
- Route decorator checks authentication (login_required) and role (role_required)
- Route function reads/writes Firestore via firebase_config.db client
- Jinja2 renders the HTML template with data from Firestore
- Response sent back to the browser

3.3 Security Model

- All passwords are managed by Firebase Auth — never stored in Firestore
- CSRF tokens protect every form submission (Flask-WTF)
- QR codes are HMAC-signed — cannot be forged without the server secret key
- Firebase Firestore rules restrict organizers to only their own events and venues
- The service account key is stored in a Docker secrets volume — never in Git
- All API keys are in .env environment variables — excluded from version control

4. Firestore Database Schema

The database uses 6 top-level collections and 1 subcollection. All designed during Day 2 based on the 20 wireframe pages of the application.

Collection 1 — users

Path: /users/{uid} | Document ID = Firebase Auth UID

Field	Type	Description
uid	string	Firebase Auth UID — matches the document ID
email	string	Login email — unique across all users
role	string	Values: attendee | organizer | admin
is_active	boolean	False = suspended account (admin action)
networking_opt_in	boolean	True = visible in attendee networking directory
created_at	timestamp	Account creation timestamp

Collection 2 — venues

Path: /venues/{venue_id} | Auto-generated document ID

Field	Type	Description
name	string	Venue name e.g. City Hall, Udupi
address	string	Street address
city	string	City — shown on event listing cards
capacity	number	Maximum attendee count
contact_name	string	Venue contact person (optional)
contact_phone	string	Venue contact number (optional)
organizer_uid	string	Owner UID — only this organizer can edit
created_at	timestamp	Server timestamp on creation

Collection 3 — events (+ ticket_types subcollection)

Path: /events/{event_id} | Subcollection: /events/{event_id}/ticket_types/{tt_id}

Field	Type	Description
name	string	Event name e.g. TechConf 2026

description	string	Full event description
start_datetime	timestamp	Event start — date + time
end_datetime	timestamp	Event end — date + time
venue_id	string	Reference to /venues/{venue_id}
event_type	string	Values: physical | virtual | hybrid
status	string	Values: draft | published | cancelled | completed
organizer_uid	string	Owner — only this organizer can manage
total_registrations	number	Atomically incremented on each confirmed registration
total_revenue	number	Sum of confirmed payment amounts
total_checkins	number	Atomically incremented on each check-in
created_at	timestamp	Server timestamp on creation
[Subcollection] name	string	Ticket type name e.g. Early Bird
[Subcollection] price	number	Price in INR — 0 for free tickets
[Subcollection] quantity_total	number	Total tickets available
[Subcollection] quantity_sold	number	Atomically incremented on each purchase
[Subcollection] max_per_order	number	Maximum a single attendee can buy
[Subcollection] is_active	boolean	False = hidden from registration page

Collection 4 — registrations

Path: /registrations/{registration_id} | Spans pages 3, 4, 5, 9, 10, 11, 12

Field	Type	Description
event_id	string	Reference to /events/{event_id}
attendee_uid	string	Reference to /users/{uid}
attendee_name	string	Denormalised for fast display
attendee_email	string	Used for manual check-in search
ticket_type_id	string	Reference to ticket_types subcollection
ticket_type_name	string	Denormalised — stored for display without join
quantity	number	Number of tickets purchased
total_amount	number	Amount actually charged after discount

status	string	Values: pending | confirmed | cancelled | checked_in
payment_status	string	Values: pending | paid | refunded | failed
qr_code_url	string	Firebase Storage URL of QR code PNG
qr_payload	string	HMAC-signed string embedded in the QR code
checked_in_at	timestamp	Set at check-in — null until scanned
checked_in_by	string	UID of staff who performed check-in
created_at	timestamp	Registration creation timestamp

Collections 5 & 6 — promo_codes and sessions

Collection	Key Fields	Purpose
promo_codes	code, discount_type, discount_value, valid_from, valid_until, max_uses, used_count	Promo codes scoped to specific events. Built in Sprint 2.
sessions	event_id, title, track, start_datetime, end_datetime, speaker_uids, streaming_url	Agenda sessions for multi-track events. Built in Sprint 3.

5. Features Built — Days 1 to 8 (Sprint 1 Complete + Sprint 2 Started)

The project follows a 14-day development plan across 2 sprints. As of the date of this document, 8 of 14 days are complete — Sprint 1 is fully done and Sprint 2 is 50% complete.

Day 1
Week 1
✓ Complete

Project Scaffolding & Docker Setup

Set up the Git repository with branch strategy, created the complete project folder structure, wrote the Dockerfile and docker-compose.yml, configured environment variable handling, and verified the Flask development server runs inside a Docker container. The foundation everything else is built on.

Wireframe: Not applicable — infrastructure setup

Day 2
Week 1
✓ Complete

Firebase Integration & Firestore Schema

Created the Firebase project, enabled Firestore (asia-south1), Firebase Auth (email/password), and designed the complete 6-collection Firestore schema mapped from all 20 wireframe pages. Wrote the firebase_config.py module and verified read/write through a /test-firebase smoke test route.

Wireframe: Not applicable — backend setup

Day 3
Week 1
✓ Complete

Authentication System

Built the complete authentication system: sign up (creates Firebase Auth user + Firestore profile), login (Firebase REST API + session cookie), logout, and forgot password. Wrote the login_required and role_required route decorators. Built base.html with role-aware navbar and all 3 auth HTML templates.

Wireframe: Wireframes — Auth pages (Login, Sign Up)

Day 4
Week 1
✓ Complete

Venue Management

Built full CRUD for venue management with Firestore. Organizer can add, edit, and delete venues with server-side validation. Ownership check ensures organizers can only modify their own venues. Delete uses POST to prevent accidental deletions.

Wireframe: Wireframe page 7 (venue section)

Day 5
Week 1
✓ Complete

Public Event Listing & Event Detail

Replaced placeholder pages with real Firestore-driven public pages. Home page shows event cards with search, event detail page shows full event info and ticket type cards with live availability. Login gate on Register button — logged-out users prompted to login first.

Wireframe: Wireframe pages 1 & 2

Day 6
Week 1

Event Creation & Organizer Dashboard

✓ Complete	Organizer can create, edit, publish, and cancel events entirely from the web app. Save as Draft and Save & Publish buttons. Real organizer dashboard shows 5 metric cards (total events, published, registrations, revenue, check-ins) and a live events list. Validates all form inputs including date comparison. **Wireframe:** Wireframe pages 6 & 7

Day 7 Week 1 ✓ Complete	**Ticket Type Management** Organizers can add Early Bird, General Admission, VIP, and any other ticket types to their events. Full CRUD with validation — price (including free tickets), quantity, max per order. Toggle activate/deactivate without deleting. Delete blocked if any tickets already sold. Tickets appear on the public event detail page immediately. **Wireframe:** Wireframe page 8

Day 8 Week 2 ✓ Complete	**QR Code Check-In System** Built the complete onsite check-in system. Uses browser camera (html5-qrcode library) to scan QR codes. HMAC signature verification ensures QR codes cannot be forged. Manual check-in by email for fallback. Blocks double check-in attempts. Atomically increments the event check-in counter. Check-in log shows all attendees with timestamps. **Wireframe:** Wireframe page 12

6. User Interface Description

All pages share a common layout defined in base.html: a deep blue navbar at the top (showing role-appropriate links), a main content area, and a dark blue footer with About, Contact, and Terms links. The design uses Bootstrap 5 with a corporate blue and white colour scheme.

6.1 Public Pages

Home Page — Event Listing (localhost:5000)

A full-width blue hero banner with the heading Upcoming Events and a search bar. Below it, event cards are arranged in a responsive grid (1 column on mobile, 2 on tablet, 3 on desktop). Each card shows the event name, date, venue, and a 120-character description snippet. Two buttons: View Details (outline style) and Register (solid blue). Unauthenticated visitors see Login to Register instead. An empty state with a calendar icon appears if no events match the search.

Event Detail Page — Ticket Selection

Two-column layout. Left column: event cover image (or blue placeholder icon), event name, date and time, venue with city, event type badge (Physical/Virtual/Hybrid), and full description in a bordered card. Right column: sticky ticket type cards panel showing each ticket's name, description, availability count, max per order, and price in INR. Sold-out tickets are greyed out. A promo code input field sits at the bottom of the panel. The Select & Register button links to the registration flow (built Day 9).

6.2 Authentication Pages

Login Page

Centred card (max 440px wide) with the EMS logo icon, a welcome message, and fields for email, password, and a role selector dropdown (Attendee or Organizer). A Forgot Password link sits next to the Password label. A Create Account link at the bottom. Flash messages appear as fixed-position dismissable alerts in the top-right corner.

Sign Up Page

Same centred card layout. Fields: email, password (minimum 6 characters), and a role selector explaining the difference between Attendee and Organizer. On successful signup the user is immediately logged in and redirected to the appropriate dashboard.

Forgot Password Page

Minimal centred card. Single email field with a Send Reset Link button. After submission, the form is replaced with a green success message regardless of whether the email exists (security best practice to prevent email enumeration).

6.3 Organizer Pages

Organizer Dashboard (localhost:5000/organizer/dashboard)

Welcome message showing the organiser's first name. Five metric summary cards in a row: Total Events, Published, Registrations, Revenue (in INR), and Check-ins — each with a Bootstrap icon and bold number. Below this, a list of event cards each showing the event name, status badge (colour-coded: green=published, grey=draft, red=cancelled), and per-event registration, revenue, and check-in counts. Manage and View buttons on each card.

Create / Edit Event Form

Full-width form on a card. Fields: Event Name, Description (textarea), Start Date & Time, End Date & Time (HTML datetime-local pickers), Venue dropdown (populated from the organiser's venues), Event Type selector (Physical/Virtual/Hybrid). Two action buttons: Save as Draft and Save & Publish. When editing, a Manage Tickets button appears in the top-right of the form header linking to the ticket management page for that event.

Venue Management

A list page showing all venues in a Bootstrap table with columns: Name & Address, City, Capacity (shown as a grey badge), Contact, and Actions (Edit and Delete buttons). Empty state with a building icon if no venues exist. The Add/Edit form has Name, Address, City, Capacity in a two-column row, and optional Contact Name and Phone fields separated by a horizontal rule.

Ticket Type Management

A table per event with columns: Name & Description, Price (green Free badge for ₹0), Total Quantity, Sold, Available (with colour-coded badges: red=Sold Out, yellow=Low Stock under 10), Status (Active/Inactive badge), and Actions. Three action icons per row: Edit (pencil), Toggle Active/Inactive (toggle icon), and Delete (trash — disabled if any sold). The Add/Edit form has Name, Description, Price, Total Quantity, and Max Per Order fields.

Check-In Scanner (localhost:5000/organizer/checkin/{event_id})

Two-column layout. Left: a live camera viewfinder powered by html5-qrcode with a square scanning overlay. A Switch Camera button for devices with multiple cameras. A result panel below the viewfinder shows success (green) or failure (red) feedback with the attendee name after each scan. Right: a Manual Entry form with an email search field and Check-In button, plus a Recent Check-Ins list that updates without page reload after each successful scan — showing the attendee name and the time of check-in.

7. Remaining Roadmap — Days 9 to 14 (MVP v1 Completion)

Sprint 2 completes the MVP v1 by Day 14. The 6 remaining days build the payment flow, QR ticket delivery, and attendee portal — the features that complete the full end-to-end event cycle from registration to check-in.

Day	Feature	What will be built	Wireframe
Day 9	**Registration Flow**	Attendee selects ticket, fills registration form, applies promo code, creates pending registration in Firestore	Page 3
Day 10	**PayPal Payment**	Payment summary page, PayPal Checkout button, webhook confirmation, payment record saved to Firestore	Page 4
Day 11	**QR Code & Email**	Generate signed QR code PNG on payment confirmation, upload to Firebase Storage, send HTML email via SendGrid with QR attached	Page 5
Day 12	**Organizer Dashboard v2**	Full registration list with search and filter, CSV export, revenue charts using Chart.js	Pages 6, 9
Day 13	**Attendee Portal**	My Events page (upcoming and past), Ticket View with large QR code display, Download Ticket and Add to Calendar buttons	Pages 10, 11
Day 14	**MVP v1 QA & Tag**	End-to-end regression testing of full flow, bug fixes, git tag v0.1.0-mvp1, README update with setup instructions	All pages

Post-MVP Phases (Weeks 3–8)

After MVP v1 (Day 14), the project continues through two more phases:

Phase 2 — Engagement (Weeks 3–4): Agenda and session management, speaker profiles, analytics charts with Chart.js, automated pre-event reminder emails, custom registration fields.

Phase 3 — Advanced (Weeks 5–8): Sponsor management module with deliverable tracking and ROI reports, post-event certificates, admin panel, CRM integration, Firestore security hardening, performance testing.

8. Development Practices

8.1 Version Control Strategy

Practice	Detail
Branch strategy	main branch is always clean and deployable. All development happens on develop branch. Feature branches used for large changes.
Daily commits	Every day's work is committed with a descriptive message e.g. 'Day 6: event CRUD, organizer dashboard with real data, publish flow'
Git tags	Each completed day is tagged e.g. v0.0.6-day6. Sprint milestones get special tags: v0.1.0-sprint1
Local backups	After each tag, a folder copy backup is maintained on D drive alongside the Git repo
Sensitive files	secrets/ folder and .env are in .gitignore — never pushed to GitHub. Verified with git status before every commit

8.2 Key Design Decisions

- Flask Blueprint pattern — separates features into independent modules, keeps app.py clean as the project scales to 20+ pages
- Firestore instead of SQL — schemaless NoSQL fits the event domain well; no migrations needed when adding fields
- Soft delete for events — setting status to cancelled instead of hard delete preserves registration records for reporting
- Atomic Firestore transactions — used for check-in counter and quantity_sold to prevent race conditions under concurrent load
- HMAC-signed QR codes — prevents ticket forgery without requiring a database lookup on every scan attempt
- Server-side validation on all forms — never trusting client-side validation alone, consistent error messaging via flash system
- Docker volumes for secrets — service account key mounted read-only into container, never copied into the image layer

8.3 Non-Functional Requirements Status

ID	Requirement	Status	Target
NFR-05	All communication over HTTPS (SSL/TLS)	Planned	Day 14
NFR-06	Passwords managed by Firebase Auth — never in Firestore	✓ **Done**	Day 2
NFR-07	No card data stored — handled entirely by PayPal	✓ **Done**	Day 10
NFR-09	Role-based route protection on all organizer/admin routes	✓ **Done**	Day 3
NFR-10	API keys in .env — never hard-coded	✓ **Done**	Day 1

NFR-11	Responsive UI on desktop, tablet, and mobile	✓ **Done**	Day 3
NFR-20	PEP 8 code standards with documentation	✓ **Done**	Ongoing
NFR-21	Git version control with clear commit messages	✓ **Done**	Day 1

9. Summary

The Event Management System is a full-stack B2B event platform built with Python Flask, Firebase Firestore, Bootstrap 5, and Docker. As of the date of this document, 8 of 14 planned development days are complete, representing the entirety of Sprint 1 and the first day of Sprint 2.

The system currently supports the complete organizer workflow from setup to event day: organizers can sign up, create venues, create and publish events, add multiple ticket types, and use the live camera-based QR check-in system. The public-facing event listing and event detail pages are fully functional, reading real data from Firestore. Authentication, role-based access control, form validation, and security best practices are all implemented and operational.

The remaining 6 days of Sprint 2 will complete the attendee-facing payment and registration flow — the last pieces needed for a fully functional MVP v1. The full roadmap then continues through Phases 2 and 3 to add agenda management, analytics, sponsor tracking, and post-event features.

8	14	6
Days Complete	**Pages Designed**	**Collections**
Sprint 1 done + Day 8	From 20 wireframe pages	Firestore database design

References

[1] Jeny, J. R. V. et al. (2022). A web-based college event management system. ICIRCA, IEEE.

[2] Liew, C.-H. & Tan, T.-Y. (2021). QR code-based student attendance system. ICSGRC, IEEE.

[3] Shah, D. A. et al. (2023). Event management systems (EMS). Journal of Applied Technology and Innovation, 7(2).

[4] Chavan, P. R. (2022). A review on Firebase for mobile application development. IJRASET, 10(1).

[5] Chen, Y.-F. et al. (2023). Using Docker container technology for rapid deployment. ICEIB, IEEE.

[6] Ferreira, H. C. et al. (2024). Accelerating cloud deployment through containerization. SMC, IEEE.